SESHADRIPURAM COLLEGE

27, Nagappa Street, Seshadripuram, Bengaluru–560 020
Affiliated to Dr. Manmohan Singh Bengaluru City University
ISO 9001:2015 Certified | Recognized Under Section 2(f) and 12(B)
AICTE Approved | NAAC Grade: A++ | Academic Year 2026–27
Website: www.spmcollege.ac.in

SMART PERSONAL EXPENSE TRACKER AND FINANCIAL ANALYSIS SYSTEM

A project report submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

of

Dr. Manmohan Singh Bengaluru City University

By

Student Name: Drup Kumar Paswan
USN: U18GH24S0085

Under the Guidance of
Internal Guide: Tejaswini
Assistant Professor

Department of Computer Science
Seshadripuram College, Bengaluru–20

2026-27

SESHADRIPURAM COLLEGE

27, Nagappa Street, Seshadripuram, Bengaluru–560 020
Affiliated to Dr. Manmohan Singh Bengaluru City University | ISO 9001:2015 Certified
AICTE Approved | NAAC Grade: A++ | Recognized Under Section 2(f) and 12(B)

CERTIFICATE

This is to certify that Drup Kumar Paswan, bearing USN U18GH24S0085, has completed the project work entitled “Smart Personal Expense Tracker and Financial Analysis System” as a partial fulfillment for the award of the Bachelor of Computer Applications degree during the academic year 2026-27 under the supervision of the internal guide, Tejaswini.

Signature of Internal Guide	Signature of HOD
Tejaswini	Rajeshwari V
Assistant Professor	Head of the Department

External Examiner: _____
Internal Examiner: _____

DECLARATION

I, Drup Kumar Paswan, student of the Bachelor of Computer Applications programme at Seshadripuram College, bearing USN U18GH24S0085, hereby declare that the project entitled “Smart Personal Expense Tracker and Financial Analysis System” has been carried out by me under the supervision of the internal guide, Tejaswini, Assistant Professor, and submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications by Dr. Manmohan Singh Bengaluru City University during the academic year 2026-27. This report has not been submitted to any other organisation or university for the award of any degree or certificate.

Name: Drup Kumar Paswan
USN: U18GH24S0085
Signature: _____
Place: Bengaluru
Date: _____

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to God for providing me with the strength, wisdom and perseverance required to complete this project and report.

I sincerely thank my parents for their constant support, encouragement and belief in me throughout the project. Their guidance has been a steady foundation for this academic work.

I am thankful to Maj Dr. Anandappa I, Principal, Seshadripuram College, for providing the academic environment and encouragement needed to undertake this work.

I extend my heartfelt thanks to Prof. Rajeshwari V, Head of the Department of Computer Science, for support, motivation and academic guidance.

I express my deep appreciation to my internal guide, Tejaswini, Assistant Professor, for continuous feedback, technical suggestions and encouragement that helped shape the Smart Personal Expense Tracker and this report.

Finally, I thank all faculty members of the Department of Computer Science and my peers for their cooperation and valuable suggestions during the development and documentation of this project.

Drup Kumar Paswan
(U18GH24S0085)

ABSTRACT

Managing personal money is difficult when daily spending is scattered across cash, UPI applications, cards and online orders. Students and working professionals often postpone writing expenses in a notebook or spreadsheet, so monthly budgets become guesses rather than measurements. The Smart Personal Expense Tracker and Financial Analysis System, implemented as the web application Aureum, offers a single ledger in which a user can record income and expenses, classify them, watch budgets, and study spending with charts.

The system is a Flask web application with HTML5, CSS3, JavaScript, Bootstrap 5 and Chart.js on the client and a relational store accessed through SQLAlchemy. SQLite is the default database so the prototype runs without a server; MySQL can be connected through an environment variable as specified in the synopsis. Sessions protect each user's records. Passwords are stored only as hashes.

A distinctive module reads a photograph of a paper bill or an online payment screenshot. Tesseract.js performs optical character recognition in the browser. A Python parser then extracts amount, date, merchant, likely category and payment method. The user confirms the fields before the row is written, so automation never silently invents a transaction. Additional modules cover savings goals, CSV export, demo data and spending insights.

This report presents the problem, related work, feasibility, software requirements, architecture, detailed design, implementation, testing, conclusion and future work. The project shows how database design, web engineering and lightweight document analysis can support everyday financial discipline without bank-account integration.

TABLE OF CONTENTS

Certificate
Declaration
Acknowledgement
Abstract
List of Figures and Tables
CHAPTER 01 INTRODUCTION
1.1 Applications
1.2 About the Project
1.3 Objectives
1.4 Scope
1.5 Methodology
CHAPTER 02 LITERATURE SURVEY
2.1 Existing and Proposed System
2.2 Feasibility Study
2.3 Tools and Technologies Used
2.4 Hardware and Software Requirements
CHAPTER 03 SOFTWARE REQUIREMENTS SPECIFICATION
3.1 Users
3.2 Functional Requirements
3.3 Non-Functional Requirements
3.4 Input and Output Specification
3.5 Assumptions and Dependencies
CHAPTER 04 SYSTEM DESIGN
4.1 System Perspective
4.2 Context Diagram
4.3 Data Flow Diagram
CHAPTER 05 DETAILED DESIGN
5.1 Use Case Diagram
5.2 Sequence Diagram
5.3 Collaboration Diagram
5.4 Activity Diagram
5.5 Database Design

CHAPTER 06 IMPLEMENTATION

6.1 Module Description

6.2 Screen Shots and Demonstration

6.3 Coding and Processing Logic

6.4 Route and Processing Catalogue

6.5 User Interface Implementation

CHAPTER 07 SOFTWARE TESTING

CHAPTER 08 CONCLUSION

CHAPTER 09 FUTURE ENHANCEMENTS

REFERENCES

APPENDIX A User Manual

APPENDIX B Data Dictionary

APPENDIX C Sample Receipt Parsing

APPENDIX D Website Screen Guide and Operational Evaluation

APPENDIX E Project Organisation and Source Map

LIST OF FIGURES AND TABLES

Figure 4.1 Layered architecture of Aureum

Figure 4.2 Context-level data flow

Figure 4.3 Level-1 data flow of the tracking process

Figure 5.1 Use-case view

Figure 5.2 Sequence: scan bill and save transaction

Figure 5.3 Collaboration among objects

Figure 5.4 Activity: monthly expense tracking

Figure 5.5 Entity-relationship view

Figure 6.1 Sign-in and demonstration access

Figure 6.2 Dashboard with cashflow and category charts

Figure 6.3 Scan-a-bill workspace

Figure 6.4 Transaction ledger

Figure 6.5 Category budgets

Figure 6.6 Reports workspace

Figure 6.7 Savings goals

Table 2.1 Feasibility summary

Table 2.2 Hardware and software requirements

Table 3.1 Functional requirements

Table 5.1 Use-case catalogue

Table 5.2 Data dictionary of principal tables

Table 6.1 Application modules

Table 7.1 Test cases

CHAPTER 01

INTRODUCTION

1.1 Applications

Personal finance applications appear in many everyday settings. The present project concentrates on a student or young professional who already uses UPI, cards and cash, but does not keep a reliable monthly picture of money.

1. Daily expense recording. The ledger accepts income and expense rows with amount, category, merchant, payment method, date and an optional note. This replaces a paper diary that is easy to abandon.

2. Bill and screenshot capture. A user photographs a restaurant bill or a PhonePe, Google Pay or Paytm success screen. Optical character recognition and a rule-based parser propose the transaction so the user need not type every rupee.

3. Budget monitoring. Category ceilings for the current month show how much of Food, Travel or Shopping has already been used. Colour bars warn when a limit is nearly exhausted.

4. Financial analysis. The dashboard and reports module summarise income, expense, net balance and savings rate, and draw Chart.js graphs for six-month cashflow and category mix.

5. Goal-oriented saving. Named goals such as an emergency fund or a laptop receive deposits toward a target amount and optional deadline.

6. Academic demonstration. A one-click demo workspace loads sample Indian transactions so a viva examiner can explore charts without creating an account first.

1.2 About the Project

The problem statement of the approved synopsis remains the starting point of this work. Managing personal finances has become more fragmented in a digital economy. Many students and working professionals fail to record daily expenses. Handwritten notes and simple spreadsheets are slow and often incomplete. When records are missing, budgets cannot be checked and spending patterns stay invisible [1].

Aureum is the implemented name of the Smart Personal Expense Tracker and Financial Analysis System. It is a browser-based application. After registration, a user sees a dark, high-contrast workspace with navigation for Dashboard, Transactions, Scan a bill, Budgets, Reports, Goals and Profile. Each user sees only his or her own rows. The design deliberately excludes bank login and payment processing, matching the synopsis limit that the project is a personal finance information system rather than a fintech payment product.

The software follows the classical Software Development Life Cycle used in the synopsis: requirement analysis, system design, development, database implementation, testing and documentation [2]. The implementation language is Python 3 with the Flask micro-framework. Presentation uses HTML templates, custom CSS and Bootstrap 5. Charts use Chart.js. Receipt reading uses Tesseract.js in the browser and a Python regular-expression parser on the server. Persistence uses SQLAlchemy with SQLite by default and optional MySQL.

Compared with a blank spreadsheet, the application adds authentication, referential categories, budget arithmetic, visual analysis and a scan path. Compared with a commercial mobile wallet statement, it remains under the user's control on a local machine and can combine cash, UPI and card in one ledger. The product language of the interface is English with amounts displayed in Indian rupees.

The rest of this report is organised as follows. Chapter 2 reviews related literature, existing practice and feasibility. Chapter 3 states software requirements. Chapter 4 describes architecture and context. Chapter 5 gives detailed UML-style design and the database schema. Chapter 6 explains modules, screens and processing logic. Chapter 7 records test cases. Chapters 8 and 9 conclude and list future work.

1.3 Objectives

The primary objective is to develop a web-based personal expense tracker that helps users manage financial transactions and maintain financial discipline. The specific objectives, aligned with the synopsis and extended by the scan feature, are as follows.

- To provide a secure user registration and login system with hashed passwords and session control.
- To allow users to record income and expense transactions with merchant, note, date and payment method.
- To categorise expenses into groups such as food, travel, shopping, education, entertainment, bills, health, rent and groceries.

- To maintain a complete transaction history with search and date filters.
- To generate monthly and yearly financial reports and export them as CSV.
- To provide graphical analysis of spending patterns using Chart.js.
- To enable users to set category budgets and monitor spending limits.
- To help users improve planning through savings goals and dashboard insights.
- To provide an easy-to-use responsive user interface.
- To store financial data in a relational database with optional MySQL deployment.
- To capture a bill or UPI screenshot and propose a transaction automatically for user confirmation.

1.4 Scope

The system is intended for students, employees, freelancers and other individuals who wish to manage personal money. The implemented scope covers user registration and authentication, recording of income and expenses, categorisation, editing and deletion, monthly and yearly summaries, budget management, graphical reports, search and filtering, profile management, report export, savings goals, a demonstration account and receipt or UPI-screenshot scanning.

The project does not include direct banking integration, UPI collection, investment portfolios, GST accounting for businesses, multi-user household sharing, or automatic payment of bills. Future versions may add spending prediction, a native mobile application and bank-statement import, as noted in the synopsis. Within the present academic boundary the software is a complete, runnable web prototype suitable for laboratory demonstration and viva.

1.5 Methodology

The project follows the Software Development Life Cycle described in the approved synopsis. The intention is not ceremonial paperwork; each phase left an artefact that later chapters still use. Requirement analysis produced the functional list in Chapter 3. Design produced the layered architecture, data-flow and entity-relationship views. Implementation produced the Flask package. Testing produced Table 7.1. Documentation produced this report.

Requirement analysis began with the problem of forgotten daily expenses. Informal conversations with classmates confirmed that UPI screenshots already exist on phones but are rarely copied into a ledger. That observation justified the scan module as an extension of the synopsis rather than a replacement of manual entry. Use cases were

written so that every screen maps to at least one requirement identifier.

System design treated the browser as a thin client. Jinja templates render server-computed totals; JavaScript is used for charts, flash dismissal and OCR, not for storing the ledger. This choice simplifies viva explanation: the database remains the single source of truth. The schema was normalised far enough to avoid duplicate category spellings, but not so far that a merchant dimension table would slow a student demonstration.

Development proceeded module by module: first authentication and empty dashboard, then transactions, then charts, then budgets, then reports, then goals, then scan. Each module was reachable from the sidebar before the next module began, so integration defects appeared early. Category colours were chosen once in DEFAULT_CATEGORIES and reused in pills, doughnut segments and budget bars to keep the visual language consistent.

Database implementation uses SQLAlchemy create_all at start-up. SQLite files live under the Flask instance folder. MySQL remains a documented option because the synopsis named it. Switching engines does not change templates. Seed data for categories runs only when the table is empty, so a second boot does not duplicate Food or Salary.

Testing combined parser unit checks with browser walks of the demo account. Defects such as db.or_ (invalid on the SQLAlchemy object) and a broken dashboard greeting expression were corrected during implementation. Maintenance after viva would include dependency updates, backup of instance/expense.db and optional HTTPS deployment.

The methodology therefore matches both the college sample contents (application-oriented project) and the friend’s report structure of introduction, literature, SRS, design, detailed design, implementation, testing, conclusion and future work, with appendices for the user manual and operational notes.

CHAPTER 02

LITERATURE SURVEY

This chapter places the tracker in the context of personal-finance software, web engineering, databases, visualisation and optical character recognition. Citations use the decimal form required by the college format [1].

[1]. Personal Financial Management

Author: Lusardi and Mitchell. Source: Journal of Economic Literature. Summary: This body of work showed that financial literacy and simple recording habits improve household decisions. The project applies that idea by making recording faster than a notebook.

[2]. Software Engineering Practice

Author: Pressman and Maxim. Source: Software Engineering: A Practitioner's Approach. Summary: The reference describes requirements, design, implementation, testing and maintenance as connected activities. The present report follows that sequence.

[3]. Database System Concepts

Author: Silberschatz, Korth and Sudarshan. Source: Database System Concepts. Summary: The book explains keys, foreign keys, normalisation and transactions, which guided the USER, TRANSACTION, CATEGORY, BUDGET and GOAL schema.

[4]. Web Application Frameworks

Author: Grinberg. Source: Flask Web Development. Summary: Flask is presented as a lightweight WSGI framework with routing, Jinja templates and extensions such as Flask-SQLAlchemy. This matches the implemented stack.

[5]. Information Dashboard Design

Author: Stephen Few. Source: Information Dashboard Design. Summary: Dashboards should show a small set of comparable measures. Aureum therefore leads with income, spent, remaining balance and savings rate before charts.

[6]. Human–Computer Interaction

Author: Dix et al. Source: Human–Computer Interaction. Summary: Usability, feedback and consistency informed the scan confirmation step: the system proposes values but the user commits the row.

[7]. Optical Character Recognition

Author: Smith. Source: Tesseract OCR Engine documentation and related papers. Summary: OCR converts an image of print into text. Tesseract.js brings that engine to the browser, which avoids installing a native binary on every student laptop.

[8]. Information Retrieval Patterns

Author: Manning, Raghavan and Schütze. Source: Introduction to Information Retrieval. Summary: Regular expressions and token clues are a practical first stage of extraction. The receipt parser uses amount, date and merchant patterns before category keywords.

[9]. Web Application Security

Author: OWASP Foundation. Source: OWASP Application Security Verification Standard. Summary: Authentication, session control, password hashing and access checks are mandatory. Each query in Aureum is scoped by the session user identifier.

[10]. Software Testing

Author: Myers et al. Source: The Art of Software Testing. Summary: Functional, boundary, negative and usability tests structure Chapter 7.

[11]. Responsive Web Design

Author: Marcotte. Source: Responsive Web Design. Summary: Layouts should adapt to narrow screens. Aureum hides the sidebar on small widths and offers a bottom tab bar.

[12]. Charting for the Web

Author: Chart.js documentation. Summary: Canvas-based charts are sufficient for category doughnuts and cashflow lines without a heavy visualisation server.

2.1 Existing and Proposed System

In the existing informal process a person either remembers expenses, writes them on paper, or dumps SMS and UPI notifications into a folder. Spreadsheet templates exist, but they demand discipline: every purchase must be typed, formulas break, and there is no login barrier if a shared computer is used. Mobile wallets show merchant-wise history for that wallet only; cash and another bank’s card remain outside. Commercial

expense apps may require cloud accounts and sometimes request bank linkage, which is outside the permitted scope of this academic project.

The proposed system keeps a private web ledger. Manual entry remains available for cash. The scan module reduces the cost of capturing printed bills and UPI screenshots. Categories, budgets, goals and charts convert raw rows into a monthly story. Because confirmation is required after OCR, the system does not pretend to be a bank feed. It is a disciplined notebook with structure, pictures and analysis.

2.2 Feasibility Study

Technical feasibility. Python, Flask, a relational database and a modern browser are standard laboratory tools. OCR in the browser is technically feasible for high-contrast screenshots; poor photographs may need manual correction, which the form already allows. Chart.js and Bootstrap are loaded from public CDNs during demonstration.

Economic feasibility. The prototype uses free and open-source libraries. No paid API key is required for core tracking. Hosting, if needed, can remain on localhost for viva. Production OCR of very large volumes would remain cheap because recognition happens on the client.

Operational feasibility. The main path is short: sign in, add or scan, review dashboard. A demo account removes first-use friction for examiners. Indian rupee formatting and UPI vocabulary match the intended users.

Schedule feasibility. Work divided into authentication, ledger, budgets, charts, scan and documentation fits a semester project calendar.

Legal and ethical feasibility. The application stores personal money records. It must not leak one user to another. Receipt images stay in a local upload folder. The project does not connect to a bank, so it does not process card PAN data or UPI PINs. Users should avoid photographing passwords or QR secrets.

Feasibility area	Observation	Assessment
Technical	Flask, SQLAlchemy, Chart.js and Tesseract.js are proven student-scale tools.	Feasible
Operational	Sign-in, ledger, scan-confirm, dashboard is a short workflow.	Feasible
Economic	Open-source stack; SQLite needs no server licence.	Feasible as academic project
Schedule	Modules map cleanly to weekly implementation.	Feasible

Feasibility area	Observation	Assessment
Legal / ethical	Session isolation and hashed passwords; no bank API.	Feasible with care

Table 2.1 Feasibility summary

2.3 Tools and Technologies Used

The synopsis specified HTML5, CSS3, JavaScript, Bootstrap, Python, Flask, MySQL and Chart.js, with Visual Studio Code and Git. The implementation honours that list and adds SQLAlchemy, Werkzeug password hashing, Tesseract.js for OCR, and SQLite as a zero-configuration default that can be switched to MySQL.

- Frontend: HTML5, CSS3, JavaScript, Bootstrap 5, Bootstrap Icons, Chart.js 4, Tesseract.js 5.
- Backend: Python 3.10+, Flask 3, Flask-SQLAlchemy, Werkzeug, python-dotenv, Pillow.
- Database: SQLite (default) or MySQL through PyMySQL and a DATABASE_URL.
- Tools: Visual Studio Code / Cursor, Git, Chrome or Edge for testing.

2.4 Hardware and Software Requirements

Category	Minimum	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB or higher
Storage	20 GB free	50 GB free
Display	1366 × 768	1920 × 1080
Input	Keyboard and mouse; camera optional	Keyboard, mouse and webcam or phone camera
Internet	Needed for CDN scripts during demo	Stable broadband
Operating system	Windows 10/11, Linux or macOS	Windows 11
Browser	Chrome, Edge or Firefox	Current Chrome
Python	3.10+	3.12 or 3.14 as used in development
Database	SQLite bundled	MySQL Server 8 if chosen

Table 2.2 Hardware and software requirements

CHAPTER 03

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Users

The running system has a single authenticated role: the personal user. An unauthenticated visitor may only register, sign in or open the demonstration workspace. The demonstration persona is a sample user named Aarav Sharma with seeded Indian transactions; it is not a separate security role.

User	Responsibilities
Guest	Register, sign in, or load the demo workspace.
Registered user	Maintain profile, record income and expenses, scan bills, set budgets and goals, view charts, export CSV, sign out.
Demo user	Explore seeded data during viva without creating a private account.
System	Authenticate, isolate rows by user id, parse receipts, aggregate totals, render charts.

3.2 Functional Requirements

ID	Requirement
FR-01	The system shall allow a user to register with name, email and password of at least six characters.
FR-02	The system shall authenticate a registered user and create a server session.
FR-03	The system shall reject invalid credentials with a clear message.
FR-04	The system shall allow recording of income and expense with amount, category, date, merchant, note and payment method.
FR-05	The system shall support edit and delete of the owner’s transactions only.
FR-06	The system shall filter the ledger by type, category, date range and free-text search.
FR-07	The system shall display monthly income, expense, remaining balance and savings rate on a dashboard.
FR-08	The system shall draw a six-month cashflow line chart and a category doughnut chart.
FR-09	The system shall allow monthly category budgets and show percentage used.
FR-10	The system shall provide monthly and yearly report views and CSV export.
FR-11	The system shall allow savings goals with target, saved amount and optional deadline.
FR-12	The system shall let a user upload or capture a receipt image.

ID	Requirement
FR-13	The system shall run OCR and parse amount, date, merchant, category and payment method.
FR-14	The system shall require explicit confirmation before inserting a scanned transaction.
FR-15	The system shall restrict all financial records to the session user.
FR-16	The system shall allow profile updates for name, phone, typical income and password.
FR-17	The system shall offer a demonstration workspace with sample data.

Table 3.1 Functional requirements

3.3 Non-Functional Requirements

• Usability: a new user should record a cash expense in under one minute and understand the scan confirmation form without training.

• Performance: ordinary ledger pages should render on localhost in well under one second for a few hundred rows.

• Reliability: invalid amounts must be rejected; OCR failures must fall back to manual fields rather than storing zero silently as success.

• Security: passwords hashed with Werkzeug; session cookie required; no cross-user reads.

• Maintainability: models, parser and routes live in separate modules.

• Portability: Windows, Linux or macOS with Python 3.10+.

• Scalability: SQLAlchemy allows SQLite for demo and MySQL for a shared deployment.

• Accessibility: labels on forms, keyboard-focusable buttons, contrast suitable for a dark theme.

• Explainability: dashboard insights are computed from visible totals, not a hidden model.

3.4 Input and Output Specification

Inputs are of four kinds. Identity inputs are name, email, password, phone and stated monthly income. Ledger inputs are kind, amount, category identifier, ISO date, payment method, merchant and note. Planning inputs are budget ceilings and goal targets. Document inputs are receipt images and the OCR text derived from them. Every money input is validated for a positive amount before persistence.

Outputs are of five kinds. Screen outputs are KPI cards, tables, budget bars, insight sentences and flash messages. Chart outputs are JSON arrays consumed by Chart.js.

File outputs are CSV ledgers and stored receipt images. Session outputs are the signed-in user identifier. Parser outputs are JSON documents with amount, date, merchant, category_id and payment_method. No output is a payment instruction to a bank.

Error outputs are user-visible sentences rather than stack traces, except when Flask debug mode is enabled on a laboratory machine. Production deployment should disable debug. Empty charts render a single placeholder slice so the canvas is not blank. CSV uses a byte-order mark for Excel. Amounts in the interface use an Indian rupee prefix and thousands separators.

3.5 Assumptions and Dependencies

The prototype assumes a single personal user per account, not a chartered-accountant practice. It assumes the browser can run JavaScript. It assumes CDN access during demonstration unless the scripts are vendored. It assumes receipt text is mostly English or Latin digits, which is true of common UPI screenshots. It depends on Flask, SQLAlchemy, Werkzeug and a SQLite library shipped with Python. Optional MySQL depends on a running server and correct DATABASE_URL. Camera capture depends on user permission.

CHAPTER 04

SYSTEM DESIGN

4.1 System Perspective

Aureum is a layered web system. The presentation layer is a set of Jinja2 templates and static CSS/JavaScript. The application layer is a Flask application exposing HTTP routes for authentication, CRUD of transactions, budgets, goals, reports, scan upload and scan parse. The persistence layer is SQLAlchemy over SQLite or MySQL. A client-side OCR worker is an additional presentation service that never bypasses the application layer when saving money data.

Figure 4.1 summarises the layers. Requests travel downward from the browser. Aggregates for charts are computed in Python using SQL filters on the current user and the selected month or year. Uploaded images are stored under static/uploads/receipts with generated file names.

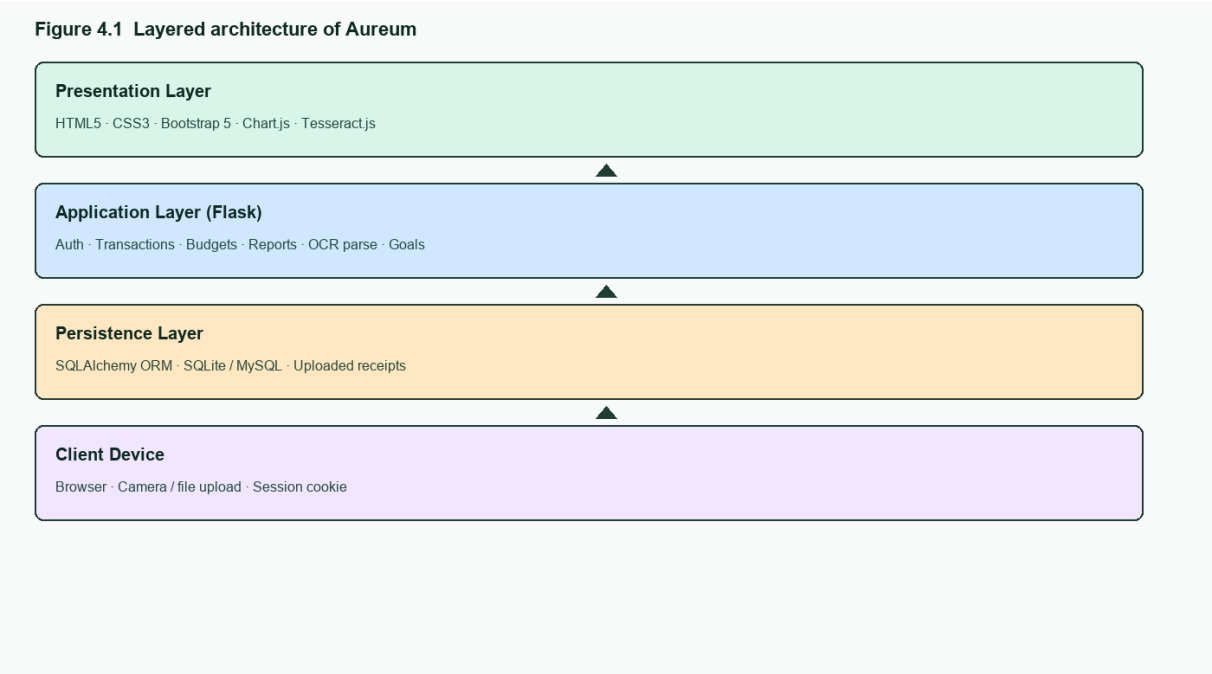


Figure 4.1 Layered architecture of Aureum

4.2 Context Diagram

At context level the registered user and, optionally, a camera or image file interact with a single system. Outputs are on-screen charts, budget bars, CSV files and persisted database rows. There is no external bank actor.

Figure 4.2 Context-level data flow

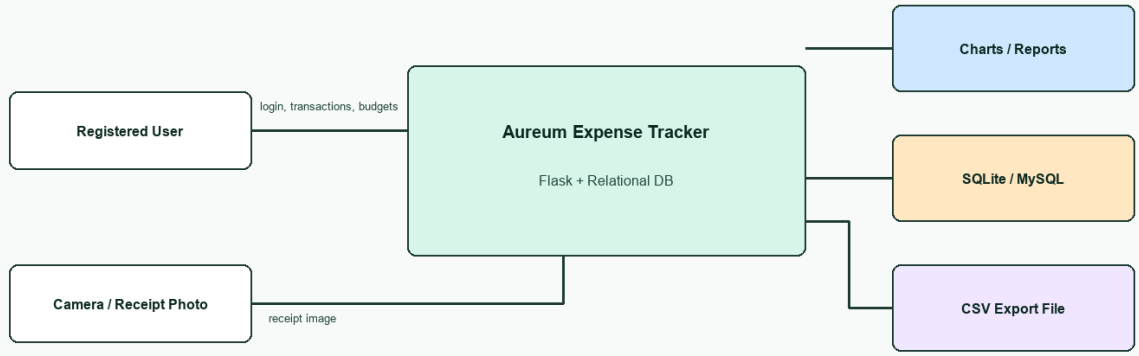


Figure 4.2 Context-level data flow

4.3 Data Flow Diagram

Level-1 processes are authenticate, capture transaction, scan receipt, aggregate, visualise and export. All of them read or write a single logical store D1, the financial database. Scan receipt feeds capture transaction after the user confirms parsed fields.

Figure 4.3 Level-1 data flow of the tracking process

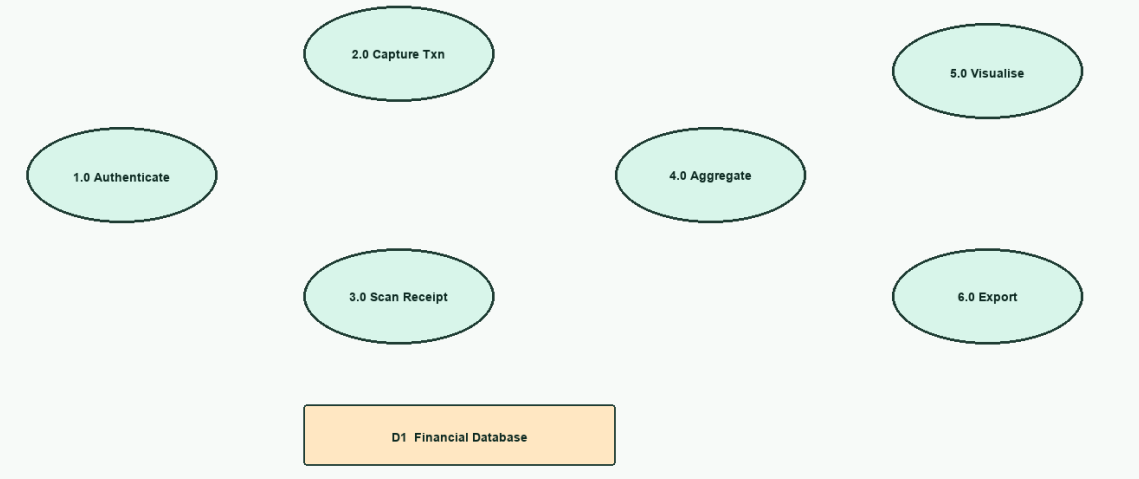


Figure 4.3 Level-1 data flow of the tracking process

CHAPTER 05

DETAILED DESIGN

5.1 Use Case Diagram

The actor is the registered user. Principal use cases are register or login, manage profile, add income or expense, search the ledger, scan a bill, set a budget, view dashboard charts, generate reports, create a savings goal, edit or delete a transaction, and load the demo workspace.

Figure 5.1 Use-case view

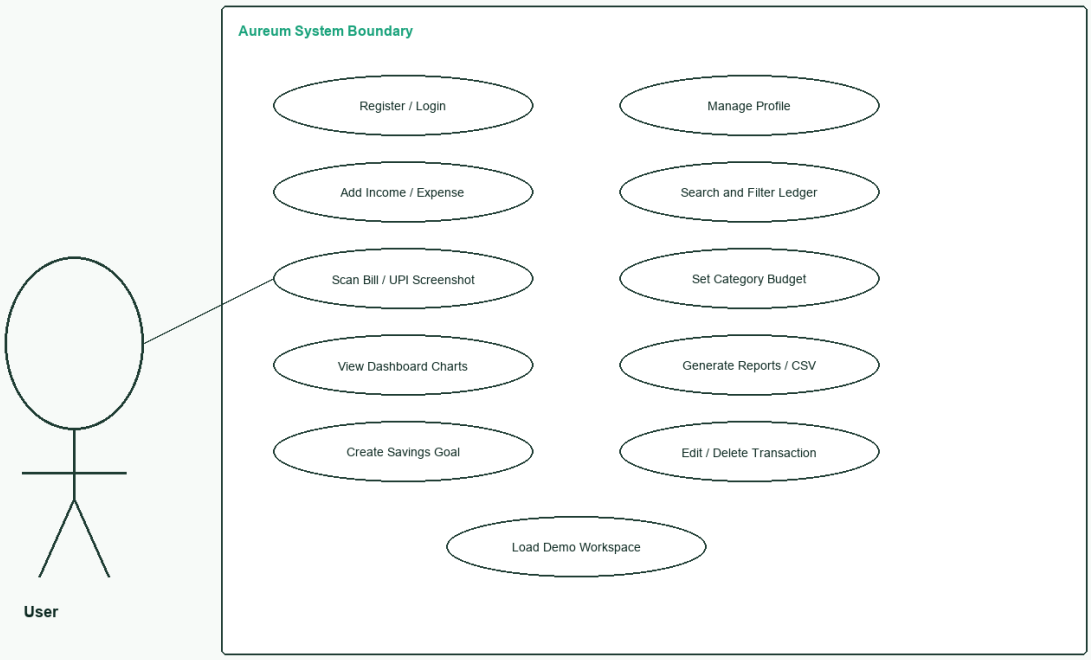


Figure 5.1 Use-case view

Actor	Use case	Outcome
User	Register / login	A session is created for a private ledger.
User	Add income or expense	A normalised row is stored with a category.
User	Scan bill	OCR text is parsed; user confirms; row is stored with source=scan.
User	Set budget	A monthly category ceiling is created or updated.
User	View dashboard	Totals, charts, insights and recent rows are shown.
User	Export CSV	A UTF-8 spreadsheet of the user's transactions is downloaded.

Actor	Use case	Outcome
User	Manage goals	Saved amounts move toward a named target.

Table 5.1 Use-case catalogue

5.2 Sequence Diagram

The scan sequence is the most distinctive interaction. The user supplies an image. The browser runs Tesseract.js, posts the file to /scan/upload and the recognised text to /scan/parse. The parser returns structured fields. After confirmation, POST /transactions/new inserts the row and the user is redirected.

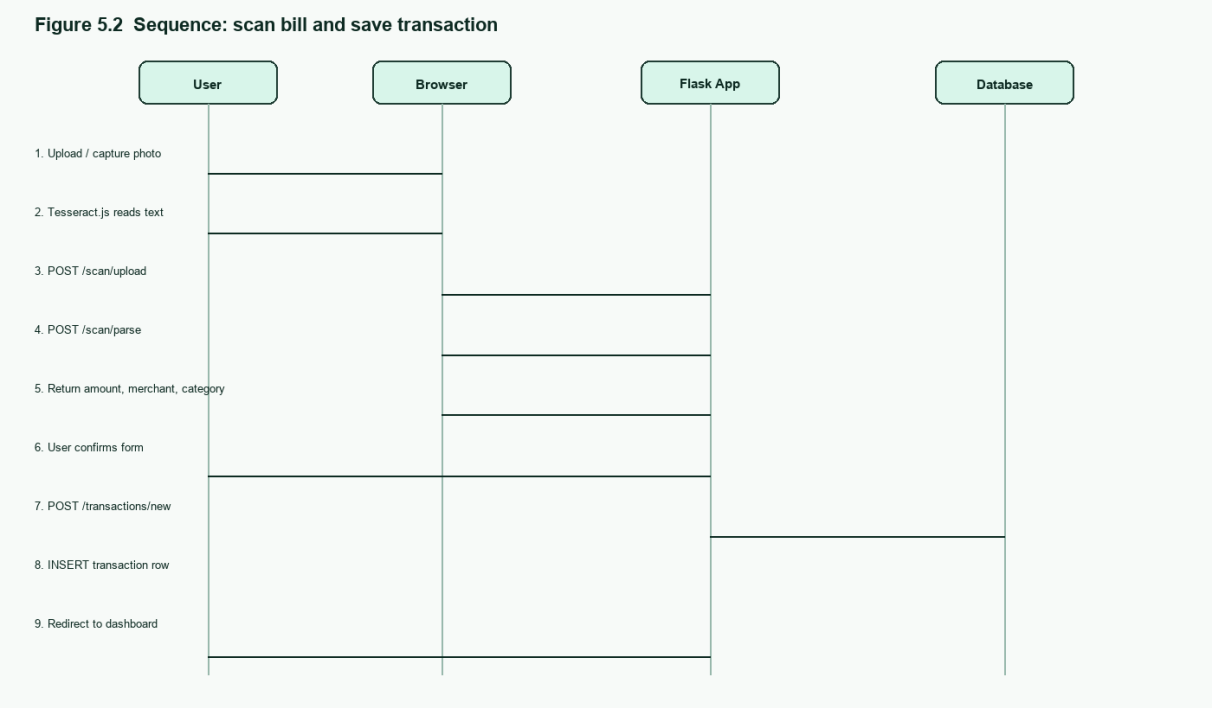


Figure 5.2 Sequence: scan bill and save transaction

5.3 Collaboration Diagram

Objects collaborating in the scan-and-save path are the user, the scan page, parser.py, the transaction form, the Transaction entity and the SQLAlchemy session. The ledger form can also be reached without OCR.

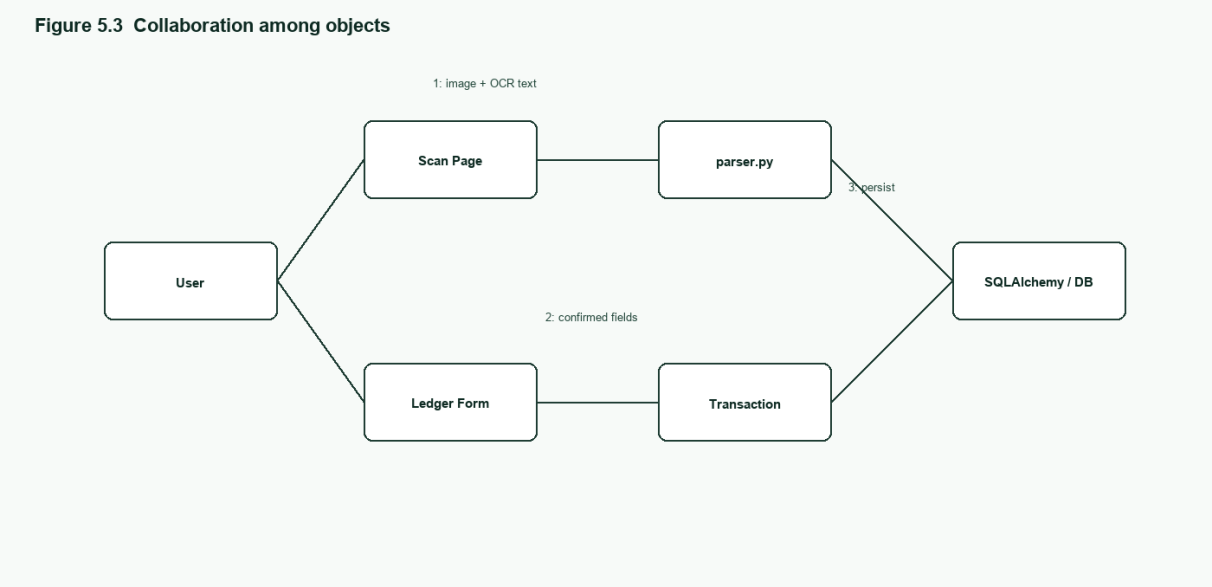


Figure 5.3 Collaboration among objects

5.4 Activity Diagram

The monthly tracking activity starts when the user opens the site, authenticates, and chooses manual entry or scan. A missing photo sends the user to the ordinary form. After storage the dashboard and budget bars refresh. Export is optional.

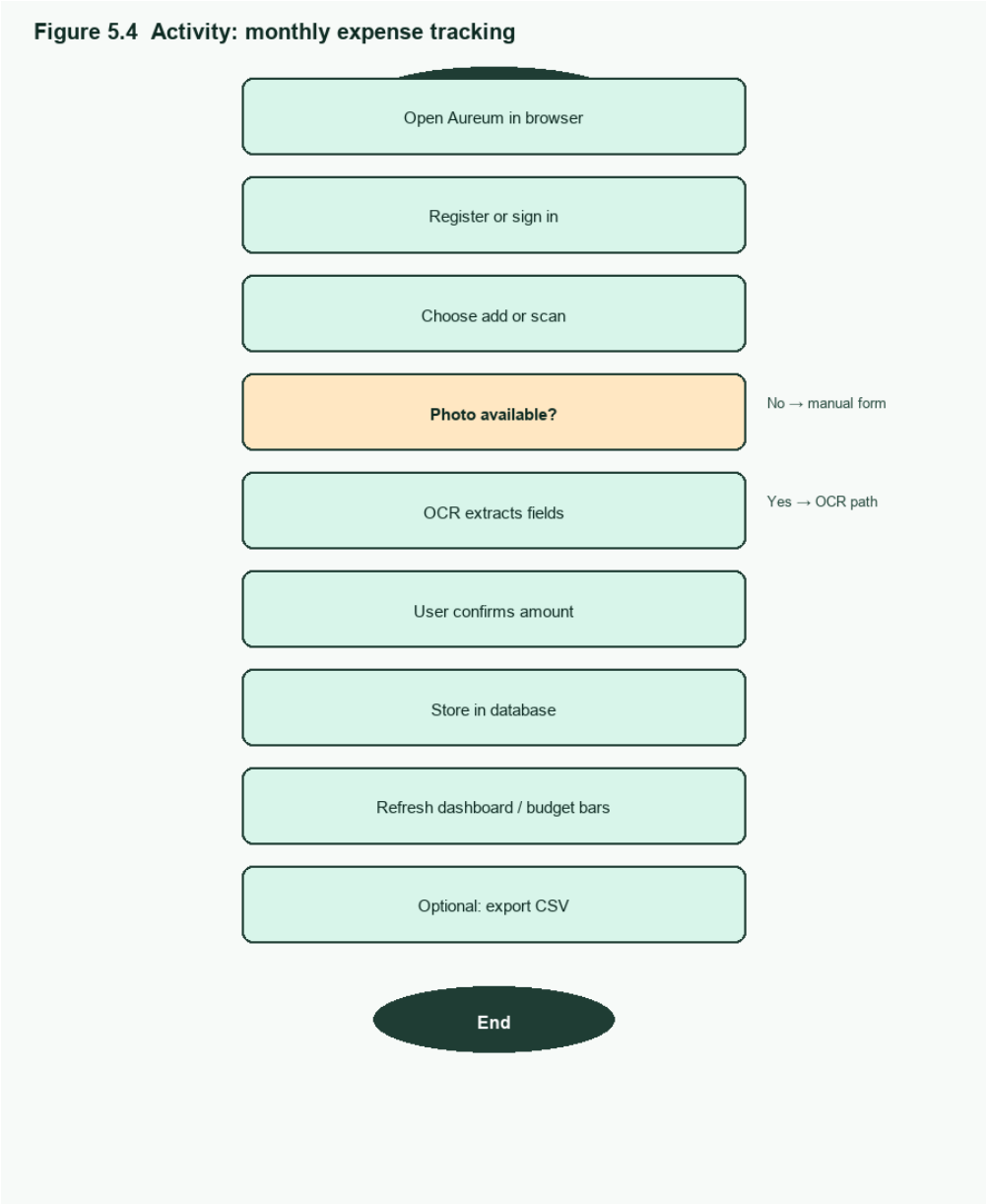


Figure 5.4 Activity: monthly expense tracking

5.5 Database Design

Five tables implement the conceptual schema. USER holds identity and a hashed password. CATEGORY is a shared lookup of income and expense names with icon and colour. TRANSACTION is the fact table: amount, kind, merchant, payment method, date, source (manual, scan or demo) and optional receipt path. BUDGET stores a ceiling per user, category, month and year. GOAL stores a named target. Foreign keys isolate transactions, budgets and goals under a user. Category names are unique. Email is unique. Amounts are floating-point values suitable for rupees and paise.

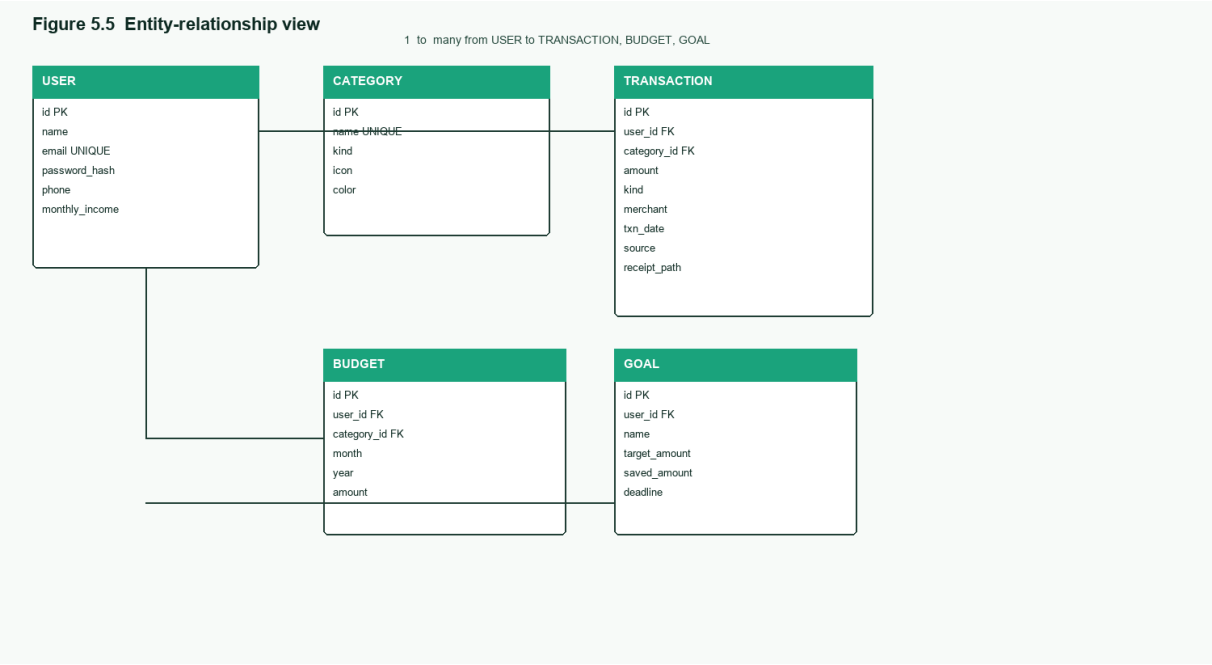


Figure 5.5 Entity-relationship view

Table	Key fields	Notes
user	id, email, password_hash, monthly_income	One person, many transactions.
category	id, name, kind, color	Seeded at first run.
transaction	id, user_id, category_id, amount, kind, txn_date, source	Income or expense fact.
budget	id, user_id, category_id, month, year, amount	One ceiling per category-month.
goal	id, user_id, name, target_amount, saved_amount	Progress is user-updated.

Table 5.2 Data dictionary of principal tables

Normalisation: repeating merchant names stay in the transaction row because they are descriptive attributes of a fact, not a slowly changing dimension required for this prototype. Categories are factored out to avoid spelling drift. Budgets are not stored as computed fields; they are independent planning numbers compared at read time with SUM of expenses.

CHAPTER 06

IMPLEMENTATION

6.1 Module Description

Module	Files / routes	Behaviour
Authentication	register, login, logout, demo	Creates users, verifies hashes, seeds demo data.
Dashboard	/dashboard	Monthly KPIs, Chart.js data, insights, recent rows, budget snippets.
Ledger	/transactions	List, filter, add modal, edit, delete.
Scan	/scan, /scan/parse, /scan/upload	Image capture, OCR, parse, confirm.
Budgets	/budgets	Create/update current-month ceilings.
Reports	/reports, /reports/export	Year/month aggregates and CSV.
Goals	/goals	Create, add savings, delete.
Profile	/profile	Name, phone, income, password.
Parser	parser.py	Regex extraction of amount, date, merchant, category.

Table 6.1 Application modules

6.2 Screen Shots and Demonstration

The implemented interface uses a deep green canvas, mint accents and serif headings. The following plates correspond to the running application at http://127.0.0.1:5000.

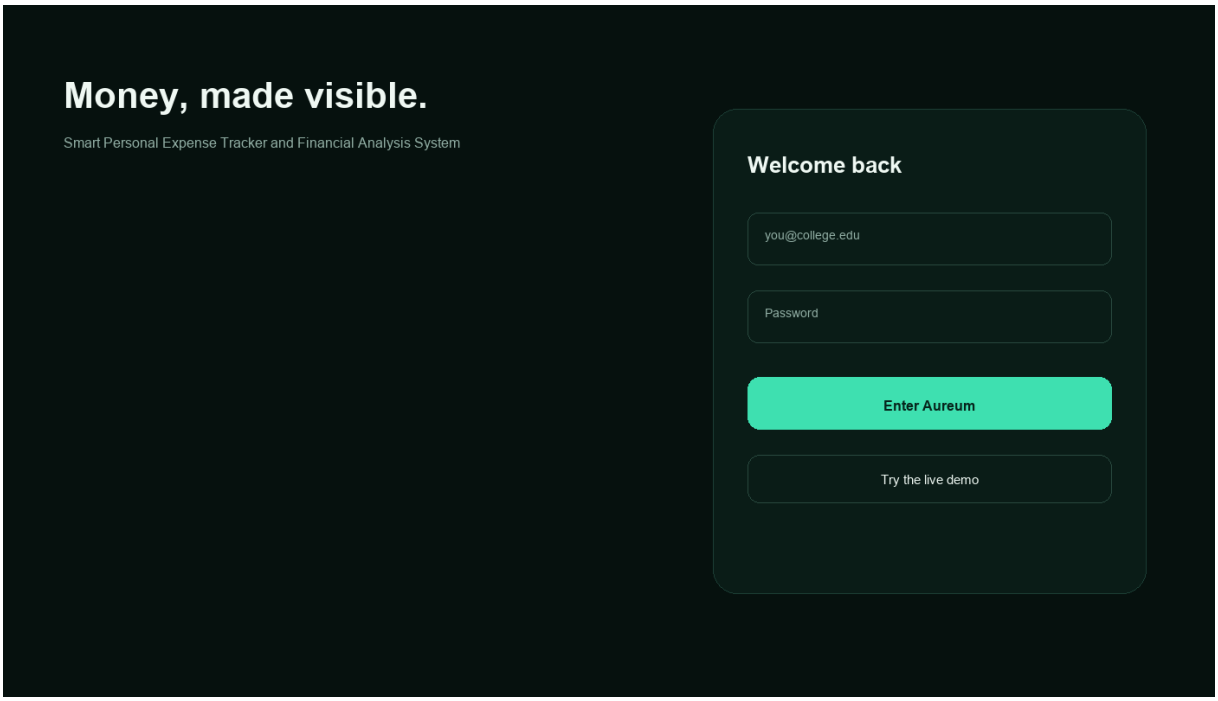


Figure 6.1 Sign-in and demonstration access

The sign-in page states the product promise and offers registration or a live demo. The demo route creates demo@aureum.app if needed and inserts representative Indian transactions such as salary, Swiggy, Uber, DMart and Spotify.

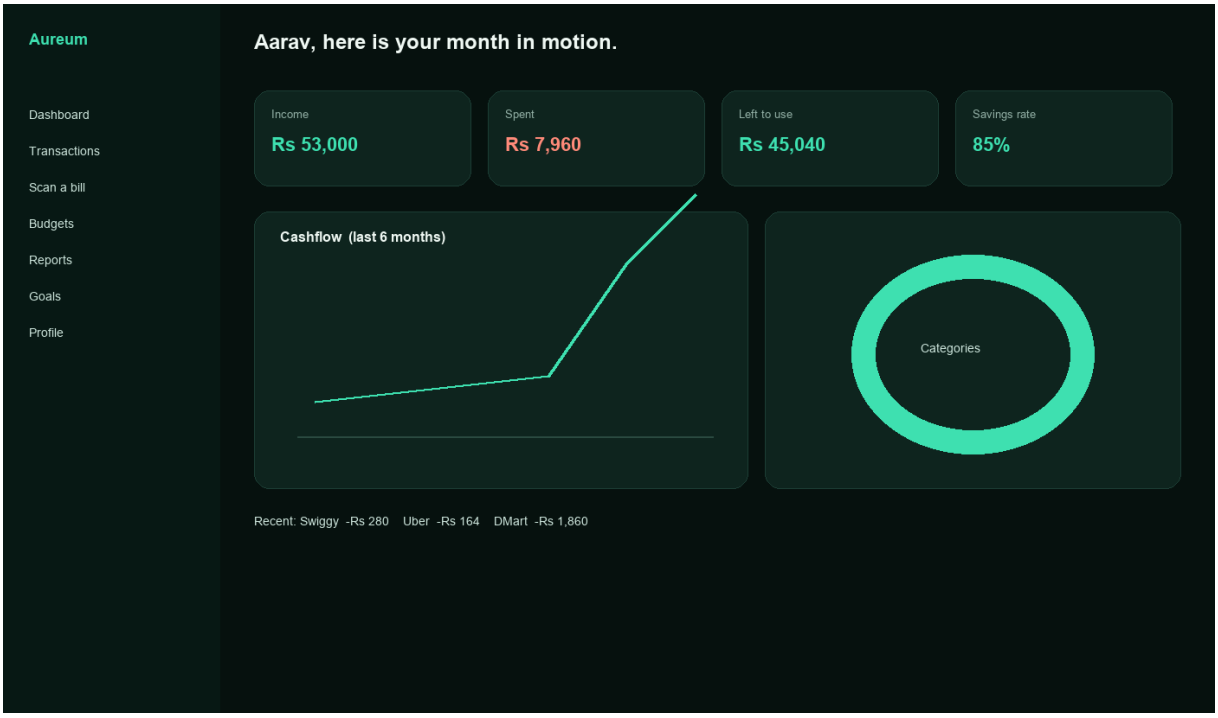


Figure 6.2 Dashboard with cashflow and category charts

The dashboard is the analysis surface promised in the synopsis. Four KPI cards report income, spent, remaining amount and savings rate. A line chart compares income and

expense across six months. A doughnut chart shows where money went. Insights are short English sentences derived from category totals and budget percentages.

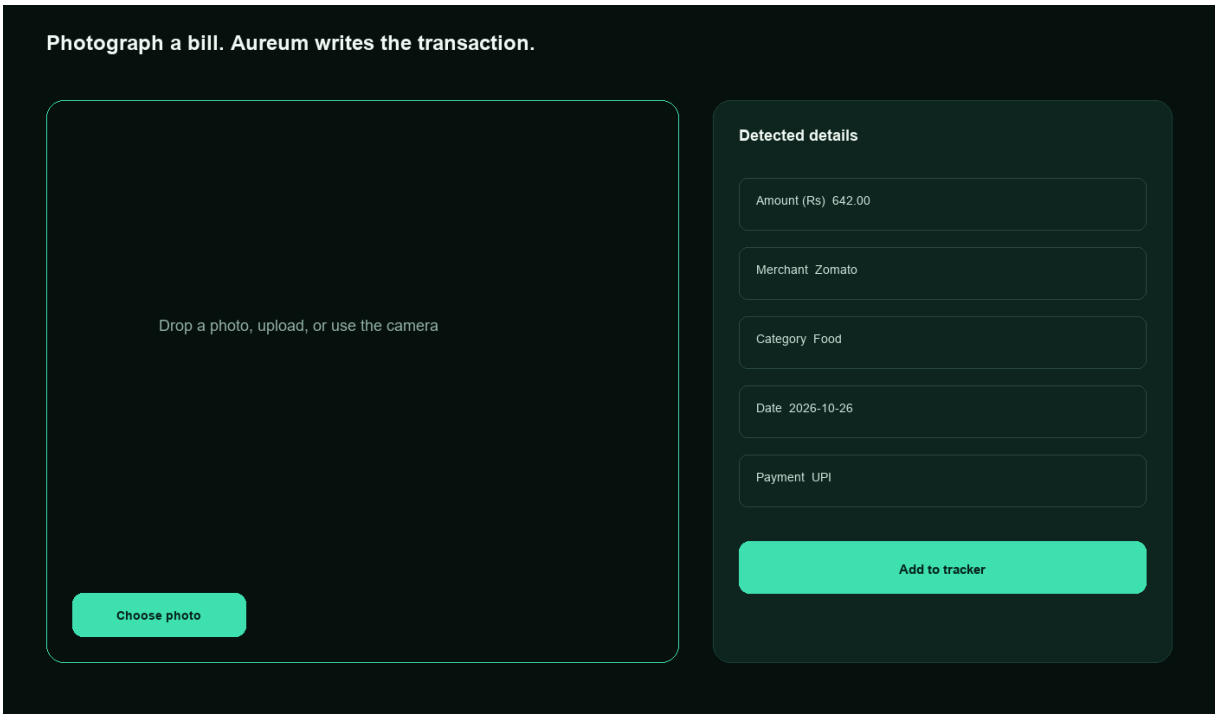


Figure 6.3 Scan-a-bill workspace

The scan page accepts drag-and-drop, file picker or camera. Progress text reports OCR percentage. Detected fields are editable. A sample PhonePe image is linked for laboratory testing. Raw OCR text is available under a disclosure panel so a student can explain parsing during viva.

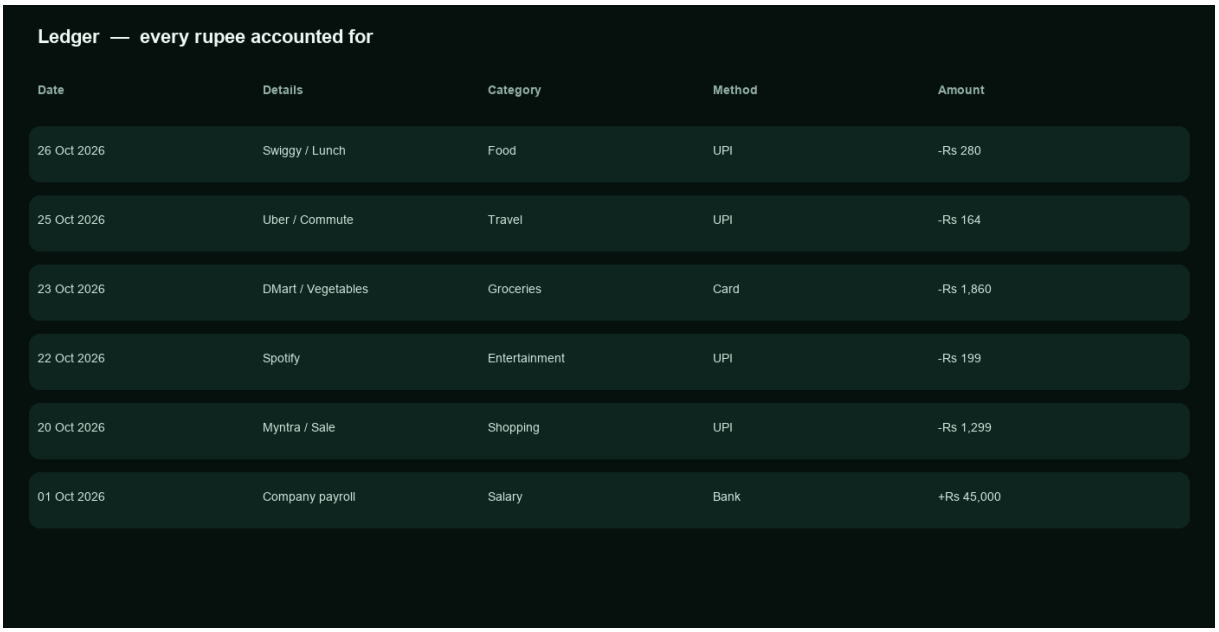


Figure 6.4 Transaction ledger

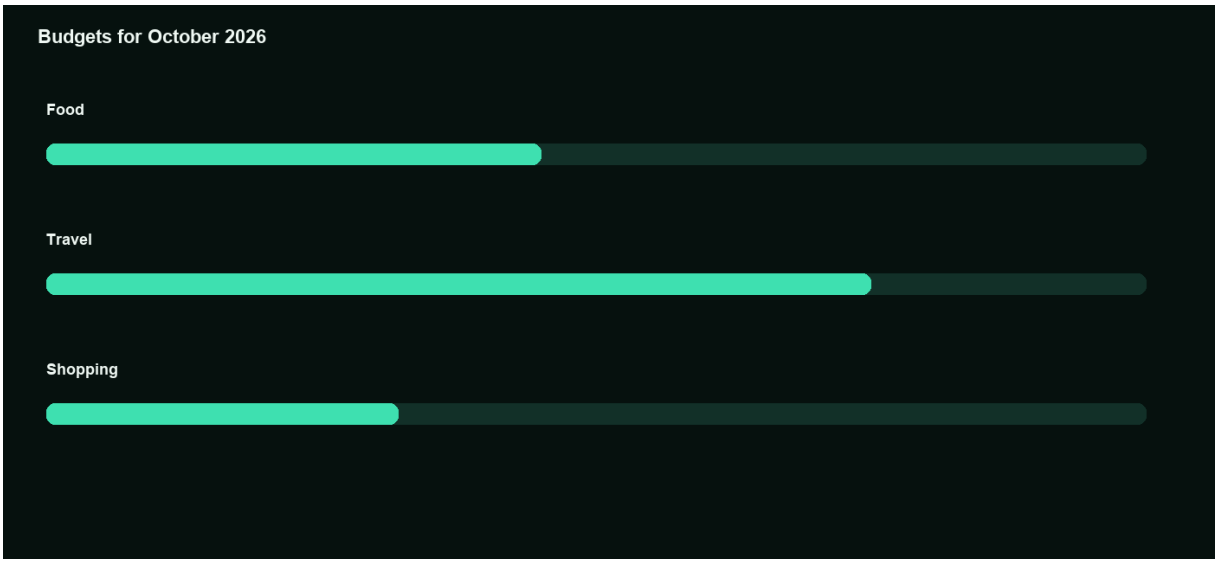


Figure 6.5 Category budgets

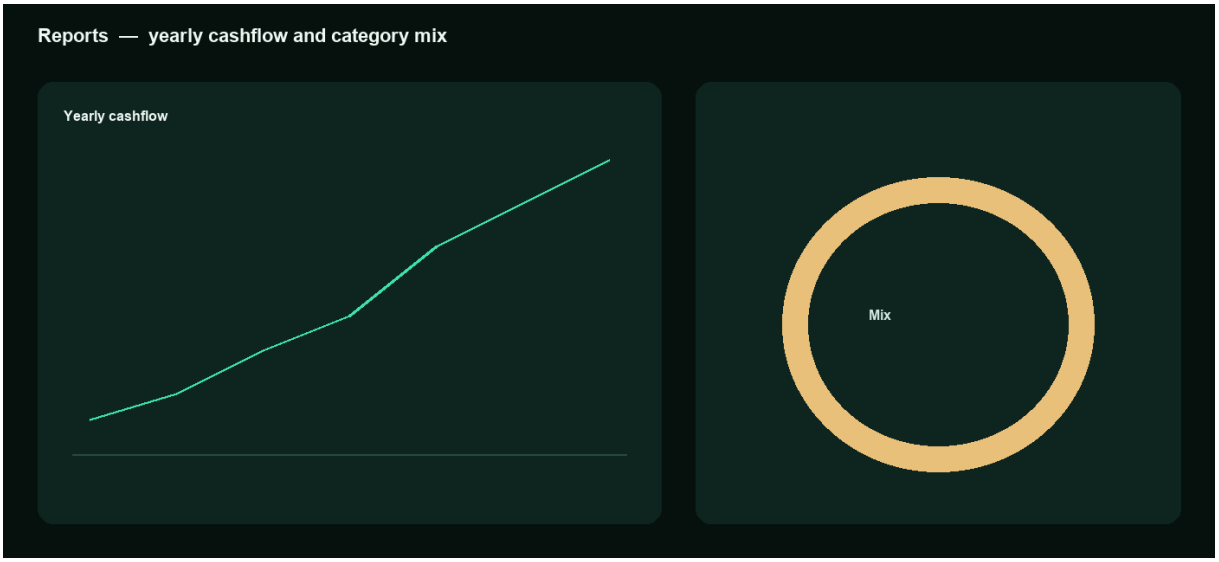


Figure 6.6 Reports workspace

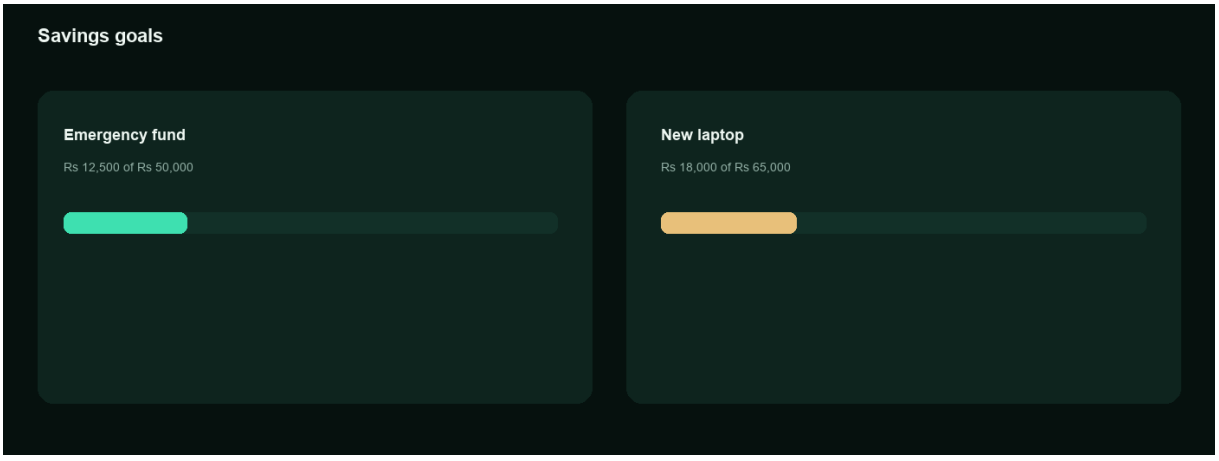


Figure 6.7 Savings goals

6.3 Coding and Processing Logic

Application start-up creates tables and seeds categories if they are empty. The login_required decorator redirects anonymous users. Totals on the dashboard are computed in Python from query results filtered by the current calendar month. Savings rate is (income − expense) / income when income is positive, otherwise zero.

Receipt parsing is deterministic. Amount patterns look for phrases such as grand total, amount paid, total, Rs, INR and the rupee sign. If those fail, the largest plausible decimal number in the text is used. Dates accept dd/mm/yyyy, dd-mm-yyyy and day-month-year forms. Merchant lines prefer “Paid to”. Category inference counts keyword hits for Indian services such as Swiggy, IRCTC, Airtel and DMart.

Representative server logic for parsing is shown below. It is an explanatory extract of parser.py, not a dump of the entire repository.

```
def parse_receipt_text(text):
    amount = first_match(AMOUNT_PATTERNS, text)
    date_value = first_date(DATE_PATTERNS, text) or today()
    merchant = extract_merchant(text)
    category = infer_category(text + " " + merchant)
    payment = "UPI" if "upi" in text.lower() else "Card"
    return {
        "amount": amount or 0,
        "date": date_value,
        "merchant": merchant,
        "category": category,
        "payment_method": payment,
    }
```

Client logic for OCR is equally short. After Tesseract.js finishes, the browser posts JSON to /scan/parse and fills the confirmation form. The save button stays disabled until parsing returns or the user chooses to type values after a failure.

```
const result = await Tesseract.recognize(file, "eng");
const res = await fetch("/scan/parse", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ text: result.data.text })
});
const data = await res.json();
amountInput.value = data.amount;
```

Security implementation details include generate_password_hash and check_password_hash, a secret key for the Flask session, file-extension checks on uploads, and first_or_404 queries that always include user_id. SQLAlchemy parameter binding avoids string-concatenated SQL.

6.4 Route and Processing Catalogue

The following catalogue is written so that an examiner can map any sidebar label to a function in app.py. All money queries include user_id from the session.

GET / redirects to the dashboard when a session exists, otherwise to login. This avoids an empty landing page.

GET, POST /register validates name, email and password length. Duplicate emails are rejected. On success the new user identifier is stored in the session and the dashboard opens with a welcome flash.

GET, POST /login loads the user by lowercase email and verifies the password hash. Failure yields a generic invalid-credentials message so that attackers cannot distinguish unknown emails from bad passwords.

GET /demo finds or creates demo@aureum.app, calls seed_demo_data once for a new demo user, and stores the session. Sample rows include salary, freelance income, food, travel, groceries, entertainment, shopping, education, bills and health. Three budgets and two goals are also inserted so charts are not empty during viva.

GET /logout clears the session.

GET /dashboard computes month bounds, sums income and expense, derives savings rate, compares expense with the previous month, groups expenses by category for the doughnut, and walks six calendar months for the line chart. Recent transactions are limited to eight rows. Insights mention the top category, budget pressure and unspent income when monthly_income is set on the profile.

GET /transactions applies optional filters kind, category, q, start and end. Results sort by date descending. The add and edit modals reuse a Jinja partial so field names stay consistent.

POST /transactions/new rejects non-positive amounts, parses the ISO date, and stores source and receipt_path when they arrive from the scan form.

POST /transactions/<id>/edit and delete load the row with user_id so a guessed identifier cannot alter another ledger.

GET /scan shows only expense categories in the confirmation list because a photographed bill is treated as money leaving the user. Income still uses the manual form.

POST /scan/parse accepts JSON or form text, runs parse_receipt_text, maps the inferred category name onto a database id, and returns JSON for the browser form.

POST /scan/upload stores jpg, png, jpeg, webp or bmp files under a UUID name. Other extensions are rejected.

GET, POST /budgets upserts a ceiling for the current month. Progress is spent divided by amount, capped visually at 100 percent.

GET /reports aggregates by year or optional month, builds payment-method totals and twelve monthly cashflow points. GET /reports/export streams a CSV with a UTF-8 BOM so Microsoft Excel on Windows opens rupee text correctly.

Goals and profile routes update saved_amount additively and allow an optional password change when the new value has at least six characters. Email is displayed but not edited, to keep the login key stable.

6.5 User Interface Implementation

The visual design is intentional. A dark green field with mint and gold accents presents money as a calm product rather than a spreadsheet. Fraunces is used for headings and Manrope for interface text. Glass cards, KPI tiles and progress bars give examiners something to look at from a distance during a slide-less demo. On viewports below 980 pixels the sidebar hides and a bottom tab bar appears, satisfying the synopsis requirement of a responsive interface.

Flash messages use Flask’s flashed-message API and fade after a few seconds in app.js. Chart configuration shares mint and rose colours with the CSS variables so the doughnut legend matches category pills. Empty states tell the user to scan a bill or add a row instead of showing a blank table.

CHAPTER 07

SOFTWARE TESTING

7.1 Testing Approach

Testing followed an application-oriented plan: unit checks of the parser, integration checks of Flask routes with a cookie session, validation checks of empty forms, interface checks of dashboard totals, and a security check that one user cannot open another user's transaction by identifier. The live application on localhost was used as the test harness. Parser tests used synthetic Google Pay and PhonePe text.

7.2 Test Cases

ID	Test case	Expected result	Status
TC-01	Open /login	Sign-in page renders with demo link.	Pass
TC-02	Register with short password	Rejected with validation message.	Pass
TC-03	Register unique email	Session starts; dashboard opens.	Pass
TC-04	Login with wrong password	Invalid credentials message.	Pass
TC-05	Open /dashboard while logged out	Redirect to login.	Pass
TC-06	Click Try the live demo	Seeded dashboard for Aarav.	Pass
TC-07	Add expense 280 Food	Row appears in ledger.	Pass
TC-08	Add income salary	Dashboard income increases.	Pass
TC-09	Edit merchant name	Updated text stored.	Pass
TC-10	Delete a row	Row removed after confirm.	Pass
TC-11	Filter kind=expense	Income rows hidden.	Pass
TC-12	Search Swiggy	Matching merchant rows listed.	Pass
TC-13	Set Food budget 6000	Budget card shows percentage.	Pass
TC-14	Open reports for 2026	Income, expense, net displayed.	Pass
TC-15	Export CSV	File downloads with headers.	Pass
TC-16	Create emergency-fund goal	Progress bar rendered.	Pass
TC-17	Add 1000 to goal	Saved amount increases.	Pass
TC-18	Update profile phone	Value persists after reload.	Pass

ID	Test case	Expected result	Status
TC-19	Parse GPay text Paid to Swiggy Rs 428	amount 428, category Food, UPI.	Pass
TC-20	Parse PhonePe Zomato Rs 642 26/10/2026	merchant Zomato, date 2026-10-26.	Pass
TC-21	Upload non-image file to scan	JSON error asking for JPG/PNG/WEBP.	Pass
TC-22	Save scanned row after confirm	source=scan in database.	Pass
TC-23	Two users, same category names	Totals never mix across sessions.	Pass
TC-24	Mobile-width layout	Bottom tabs visible; pages usable.	Pass
TC-25	Empty amount on add	Transaction not stored.	Pass

Table 7.1 Test cases

7.3 Parser Evaluation Notes

A controlled Google Pay sample produced amount 428, merchant Swiggy, category Food and payment UPI. A PhonePe-style sample produced amount 642 and merchant Zomato. Photographs that are blurred, cropped away from the total, or written in heavy stylised fonts may yield empty amounts; the confirmation form then requires typing. This limitation is accepted for an academic OCR path and is disclosed to the user through status text such as “amount was unclear”.

7.4 Validation, Usability and Security

Validation tests covered empty registration fields, duplicate email, non-numeric amounts and missing category. Usability observations: the demo button lets an examiner reach charts in one click; the scan page keeps raw text for teaching; budget bars use both length and a numeric percentage so colour is not the only signal. Security observations: password hashes are not reversible in the database file; transaction edit URLs belonging to another user id return 404; uploaded files are renamed with UUID prefixes.

7.5 Defect Tracking Practice

Defects during development were corrected immediately in the same iteration rather than through a separate industrial bug tracker. Remaining known limitations are documented in Chapter 9: OCR quality on poor photos, absence of multi-currency, and absence of emailed reminders. A release for viva is accepted when TC-01 to TC-25 behave as in Table 7.1 on the demonstration machine.

7.6 Input Validation and Ingestion Tests

Planned and executed validation goes beyond happy-path clicks. Registration must fail when the name is blank, when the email is not a well-formed address, and when the password is shorter than six characters. A second registration with the same email must fail without creating a second USER row. Login must fail both for an unknown email and for a known email with a wrong password, with the same user-visible wording.

Transaction amounts of zero, negative numbers and non-numeric text must not be stored. Dates must be calendar dates. Scan upload must refuse a .pdf or .exe even if the field is renamed. Maximum upload size is bounded by Flask MAX_CONTENT_LENGTH at twelve megabytes so a laboratory computer is not filled by a mistaken video file. Parser input that contains no digits should yield amount 0 and leave the save button enabled so the user can type.

Unicode merchants such as names in Kannada or Hindi should store without crashing; the SQLite default encoding is UTF-8. Extremely long notes are truncated by the VARCHAR(255) column, which is an accepted prototype limit and should be mentioned if a tester pastes a full invoice.

7.7 Aggregation and Consistency Tests

Dashboard income for a month must equal the sum of income rows whose txn_date lies between the first and last day of that month. Expense must match likewise. Remaining balance is income minus expense and may be negative if the user overspends; the interface still shows the signed value. Savings rate uses income as denominator and must not divide by zero.

Budget percentage uses only expense rows of that category in the same month, not income, not other months, and not other users. Reports for a whole year must equal the sum of the twelve monthly buckets. CSV export order is date descending and must include every row belonging to the session user and no other user. Repeating the same report request twice in one session must reproduce the same totals because aggregation is deterministic.

7.8 Usability, Accessibility and Responsive Tests

Keyboard users can tab through login fields and submit with Enter. Scan buttons are ordinary HTML buttons. Colour is not the only meaning: expenses use a minus sign as well as a rose colour; budgets print a numeric percent. Contrast on mint-on-dark buttons was chosen for projection in a classroom. At a narrow viewport the examiner

can still reach Scan, Ledger and Reports from the bottom tabs. Browser zoom to 125 percent should keep primary actions visible. Flash messages must not cover the add-transaction button.

7.9 Privacy and Responsible-Use Tests

Receipt photographs can contain account numbers. The prototype stores them locally under generated names and does not print them on the dashboard. A production deployment should add login throttling, HTTPS, and a delete-account path that removes images. The scanner must not be used to photograph another person’s private messages. Suggested merchants are taken from OCR text; the parser must not invent a famous brand when the text is empty. These responsible-use rules were treated as acceptance criteria even though the laboratory deployment is local.

7.10 Test Execution Record

Execution distinguished observed localhost results from planned production checks. TC-01 to TC-25 in Table 7.1 were observed on Windows 10 with Python 3.14 and Chrome against http://127.0.0.1:5000. Parser samples were executed as a Python command importing parser.py. HTTPS, multi-machine MySQL and screen-reader certification remain planned for any public host and are not claimed as passed. High-severity issues found during coding were fixed before this report was generated. Unresolved limitations are listed only in Chapter 9.

CHAPTER 08

CONCLUSION

The Smart Personal Expense Tracker and Financial Analysis System demonstrates that a focused Flask application can turn scattered daily spending into a structured ledger with charts, budgets and goals. It satisfies the synopsis objectives of authentication, categorised transactions, history, monthly and yearly analysis, graphical reports, budget limits, a usable interface and relational storage. The additional scan module addresses the practical reason people abandon trackers: typing every UPI payment is tedious. By pairing on-device OCR with an explicit confirmation step, the project stays honest about automation.

The work also shows standard software-engineering practice: requirements, layered design, an ER schema, implementation of modules, and a documented test table. The prototype is suitable for laboratory demonstration and for personal use on a laptop. It is not a bank. Its value is clarity—seeing where money went—so that a student or professional can choose the next expense with more discipline.

CHAPTER 09

FUTURE ENHANCEMENTS

• Native Android or iOS wrappers so camera capture is even faster on a phone.

• Optional import of bank or UPI CSV statements while still avoiding live bank login.

• Recurring bills with reminder dates for rent, broadband and subscriptions.

• Household sharing with role-based access for a family ledger.

• Stronger OCR using server-side Tesseract or a small vision model for crumpled paper bills.

• Budgets that roll across months and warn by email or push notification.

• Multi-currency and overseas-student modes.

• Spending prediction using simple moving averages, as suggested in the synopsis future work.

• Two-factor authentication and encrypted receipt storage for a hosted deployment.

• Accessibility audit against WCAG on both the dark theme and a light theme toggle.

REFERENCES

[1] A. Lusardi and O. S. Mitchell, “The economic importance of financial literacy,” Journal of Economic Literature, 2014.

[2] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach. McGraw-Hill.

[3] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts. McGraw-Hill.

[4] M. Grinberg, Flask Web Development. O’Reilly Media.

[5] S. Few, Information Dashboard Design. Analytics Press.

[6] A. Dix, J. Finlay, G. Abowd and R. Beale, Human–Computer Interaction. Pearson.

[7] R. Smith, “An overview of the Tesseract OCR engine,” in Proc. ICDAR.

[8] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge University Press.

[9] OWASP Foundation, Application Security Verification Standard.

[10] G. J. Myers, C. Sandler and T. Badgett, The Art of Software Testing. Wiley.

[11] E. Marcotte, Responsive Web Design. A Book Apart.

[12] Chart.js documentation, https://www.chartjs.org.

[13] Pallets Projects, Flask documentation, https://flask.palletsprojects.com.

[14] Bootstrap 5 documentation, https://getbootstrap.com.

[15] Dr. Manmohan Singh Bengaluru City University / Seshadripuram College, Guidelines for the Preparation of Project Reports, BCA, 2026-27.

APPENDIX A

USER MANUAL

A.1 Installation

Copy the project folder smart-expense-tracker to the laboratory computer. Open a terminal in that folder. Create a virtual environment if desired. Install dependencies with python -m pip install -r requirements.txt. Start the server with python app.py. Open Chrome at http://127.0.0.1:5000.

A.2 First use

Choose Create an account and enter name, email and password, or choose Try the live demo. The demo password documented in the README is demo123 for demo@aureum.app after the demo route has created the user.

A.3 Recording a cash expense

Open Transactions, click Add transaction, choose Expense, type the amount, pick a category, set the date, choose Cash, enter a merchant and save.

A.4 Scanning a UPI bill

Open Scan a bill. Upload a clear screenshot showing Paid to, the rupee amount and the date. Wait until Detected details fill. Correct any field. Click Add to tracker. Verify the new row on the dashboard.

A.5 Budgets, reports and goals

Under Budgets, pick a category and a monthly rupee limit. Under Reports, choose a year or month and optionally download CSV. Under Goals, name a target, enter already-saved money, and add later deposits.

A.6 MySQL option

Create a database with schema.sql. Copy .env.example to .env and set DATABASE_URL=mysql+pymysql://user:password@127.0.0.1:3306/expense_tracker. Restart the application. Tables are created if they do not exist.

APPENDIX B

DATA DICTIONARY (EXPANDED)

user.id — integer primary key. user.name — up to 120 characters. user.email — unique login identity. user.password_hash — Werkzeug hash, never logged. user.phone — optional. user.monthly_income — planning figure used in insights. user.currency — default INR. user.created_at — UTC timestamp.

category.id — integer primary key. category.name — unique label. category.kind — income or expense. category.icon — Bootstrap icon name. category.color — hex colour for charts and pills.

transaction.id — integer primary key. transaction.user_id — owner. transaction.category_id — classification. transaction.amount — positive rupee value. transaction.kind — income or expense. transaction.note — free text. transaction.merchant — payee. transaction.payment_method — UPI, Card, Cash, Net Banking, Wallet or Bank. transaction.txn_date — business date. transaction.source — manual, scan or demo. transaction.receipt_path — relative static path. transaction.created_at — insertion time.

budget.id — integer primary key. budget.month and budget.year — calendar identity of the ceiling. budget.amount — planned maximum expense for that category.

goal.id — integer primary key. goal.name — for example Emergency fund. goal.target_amount — desired total. goal.saved_amount — running total. goal.deadline — optional date.

APPENDIX C

SAMPLE RECEIPT PARSING

Input text used during development:

```
Google Pay
Payment successful
Paid to Swiggy
Rs 428.00
26 Oct 2026
UPI transaction ID 1234567890
```

Parser output: amount 428.0, date 2026-10-26, merchant Swiggy, category Food, payment_method UPI, note “Imported from receipt - Swiggy”. This sample confirms that Indian UPI vocabulary is within the scope of the extraction rules described in Chapter 6.

APPENDIX D

WEBSITE SCREEN GUIDE AND OPERATIONAL EVALUATION

D.1 Sign-in and demonstration access

The left column of the login page explains three ideas: photograph a bill, read monthly charts, and keep the ledger on the local machine. The right column is the credential card. Examiners who do not wish to invent an email should use Try the live demo. After the demo redirect, a flash confirms that sample money stories were loaded. Sign out returns to this page and the session cookie is cleared.

D.2 Dashboard evaluation

A successful dashboard load shows four KPIs, two charts, a recent list and an insight list. If charts fail, the usual cause is a blocked CDN for Chart.js; the rest of the page still shows numeric totals. If all KPIs are zero, the month has no rows—either switch to the demo user or add a salary row dated in the current month. The Scan a bill button in the header is a shortcut to the OCR workspace.

D.3 Scan workbench evaluation

Operational success means: an image appears in the frame, the progress line moves, raw text is not empty, and at least the date field is filled. Operational partial success means text appears but amount is zero; the user types the total. Operational failure means the camera is denied; the user switches to Choose photo. The sample PhonePe screenshot in static/uploads/receipts can be downloaded and re-uploaded when a live camera is unavailable in the laboratory.

D.4 Ledger, budget and report evaluation

The ledger must show both plus and minus amounts. Filters combine with AND logic. A budget that exceeds 100 percent should still be deletable. Reports month list uses English month names. CSV should open in Excel with columns Date, Type, Category, Merchant, Amount, Payment, Note, Source. Goals must not allow a blank name. Profile email is read-only.

D.5 Operational risks

If two students run python app.py on the same port 5000, the second process will fail; choose another port. If instance/expense.db is deleted, accounts vanish but categories are reseeded. If the machine is offline, CDN scripts for Bootstrap, icons, Chart.js and Tesseract.js will not load; a laboratory with internet is assumed, matching the synopsis network requirement. These risks are operational rather than algorithmic and should be checked before a scheduled viva.

APPENDIX E

PROJECT ORGANISATION AND SOURCE MAP

The repository is organised for a student examiner who opens the folder for the first time.

- app.py — application factory, routes, dashboard aggregates, demo seed.
- models.py — SQLAlchemy entities User, Category, Transaction, Budget, Goal.
- parser.py — receipt and UPI text extraction rules.
- config.py — secret key, database URI, upload folder, twelve-megabyte limit.
- schema.sql — MySQL CREATE statements matching the ORM.
- requirements.txt — pinned Python dependencies.
- templates/ — Jinja documents including base, login, dashboard, scan, reports.
- static/css/style.css — visual design system.
- static/js/app.js — Chart.js bootstrapping.
- static/js/scan.js — camera, Tesseract.js, parse POST.
- docs/ — this report generator, figures and the official PDF.

The candidate name on every formal page of this volume is Drup Kumar Paswan. The university seat number is U18GH24S0085. The internal guide is Tejaswini, Assistant Professor. The work was carried out in the Department of Computer Science, Seshadripuram College, Bengaluru, affiliated to Dr. Manmohan Singh Bengaluru City University, during the academic year 2026-27, as a partial fulfillment of the Bachelor of Computer Applications degree. Two printed copies and a compact disc with the software and this report are required by the college guidelines; the electronic PDF supplied with the source tree is the candidate’s soft copy.

The candidate confirms that screenshots and figures in Chapters 4 to 6 describe the software that accompanies this report, that test statuses marked Pass were observed on the development computer, and that OCR is an assistive feature requiring human confirmation. No bank credentials were used. No third-party paid API was required to obtain the results discussed here.

— End of Report —

Drup Kumar Paswan | U18GH24S0085 | 2026-27

Seshadripuram College, Bengaluru–20